

PHANTOM PROTOCOL

Technical Specification № 3

DHT-Based Trustless PKI

Zero Authority · Zero Trust · Zero Centre

March 2026 · Draft v0.1

ABSTRACT

This specification defines the Phantom Distributed Public Key Infrastructure (Phantom-DPKI): a fully decentralised, trustless PKI for Phantom Protocol that replaces directory authorities with a hardened Kademlia DHT. It specifies: (1) self-certifying node identity derivation from post-quantum key material; (2) dual-puzzle Sybil resistance combining a static key-generation puzzle and a dynamic epoch-bound puzzle; (3) the NodeDescriptor record format, its cryptographic signing, and its DHT publication lifecycle; (4) multi-path eclipse-resistant lookup; (5) a gossip-based ejection protocol for immediate propagation of misbehaviour proofs; (6) epoch-based descriptor rotation that limits the utility of compromised information; and (7) bootstrap peer discovery without any hardcoded seed nodes. Together these components ensure that the Phantom network has no identifiable authority, no legal attack surface at the PKI layer, and no single point of failure — while remaining operational even under a sustained Sybil attack controlling up to 30% of the network.

Table of Contents

1. Problem Statement — Why Directory Authorities Must Die
2. Design Goals and Security Properties

3. Node Identity — Self-Certifying Keys
4. Sybil Resistance — Dual Crypto-Puzzle Admission
5. NodeDescriptor Record Format
6. DHT Architecture — Phantom-Kademlia
7. Record Lifecycle — Publish, Replicate, Expire
8. Multi-Path Eclipse-Resistant Lookup
9. Gossip Ejection Protocol
10. Epoch Rotation and Forward Secrecy
11. Bootstrap Without Seed Nodes
12. DHT as Multi-Purpose Registry
13. Rust Implementation Guide
14. Attack Analysis
15. Open Problems
16. References

1. Problem Statement — Why Directory Authorities Must Die

Every significant mix network deployed to date — Tor, Katzenpost, Nym — relies on a small set of directory authorities: trusted servers that maintain and sign the list of active relay nodes. This architecture has three fatal properties in a GPA threat model.

1.1 Legal Compulsion

Directory authorities are identifiable legal entities. They have operators, jurisdictions, and physical infrastructure. A nation-state adversary can compel them with court orders, national security letters, or extrajudicial means. When a directory authority is compelled, the adversary gains the ability to inject malicious nodes, selectively remove honest nodes, or simply shut down the network. The Tor Project's directory authorities are publicly known. This is not a theoretical risk.

1.2 Single Point of Failure

Tor operates with approximately 10 directory authorities. A distributed denial-of-service attack against all 10 simultaneously would prevent new clients from bootstrapping. Existing circuits continue to function, but no new anonymous connections can be established. Phantom Protocol must be operational even when every previously known server is offline.

1.3 Trust Assumption Mismatch

The Phantom threat model assumes every node, every authority, and every third party may be adversarial. A PKI that requires trusting 10 specific servers is architecturally incompatible with this threat model. The trust assumption must be reduced to zero identifiable trusted parties.

The Requirement

Phantom-DPKI must provide a fully functional public key infrastructure — node discovery, key distribution, misbehaviour detection — with zero directory authorities, zero seed nodes, zero legal attack surface, and zero single point of failure. The network must bootstrap from nothing and remain operational under sustained adversarial pressure.

2. Design Goals and Security Properties

Property	Requirement	Mechanism
Decentralisation	No identifiable trusted party at any level	Kademlia DHT, no seed nodes
Self-certification	Node identity verifiable without any authority	Identity = H(pubkeys)
Sybil resistance	Cost to create N identities scales linearly	Dual PoW crypto-puzzle
Eclipse resistance	Lookup succeeds despite adversarial routing	Multi-path disjoint lookup
Misbehaviour detection	Invalid proofs eject nodes in < 5 seconds	Signed gossip ejection
Forward secrecy	Past descriptors useless after epoch end	1-hour epoch rotation
GPA resistance	NSA-level observer cannot map node IPs to IDs	Sphinx ⁺ over QUIC, no cleartext IDs
Bootstrap resilience	Network starts from zero known peers	DNS-seeded + local mDNS + hardcoded genesis CID
Post-quantum	Key material safe against quantum adversary	Kyber-1024 + Dilithium-3

Table 1 — Design goals and their mechanisms

3. Node Identity — Self-Certifying Keys

A Phantom node's identity is derived entirely from its cryptographic key material. No registration with any external party is required or possible. Any network participant can verify any node's claimed identity using only the node's public keys and a hash function — no PKI lookup, no certificate chain, no trusted root.

3.1 Key Generation

```
// Step 1: Generate long-term key pairs
let sk_ed    = Ed25519::generate(&mut rng);    // classical signing
let sk_dil   = Dilithium3::generate(&mut rng); // PQ signing
let sk_x25   = X25519::generate(&mut rng);    // classical KEM
let sk_kyber = Kyber1024::generate(&mut rng);  // PQ KEM

// Step 2: Solve static admission puzzle (once per identity)
// Find (sk_ed, pk_ed) such that H(pk_ed) has c1 leading zero bits
// This is the S/Kademlia static puzzle — prevents free identity creation
loop {
    let (sk, pk) = Ed25519::generate(&mut rng);
    if blake3(pk.as_bytes()).leading_zeros() >= C1_BITS { // C1 = 20 bits
        sk_ed = sk; break;
    }
}

// Step 3: Derive node_id (self-certifying)
let node_id = blake3(
    b"phantom-node-v1"
    || pk_ed.as_bytes()           // 32 bytes
    || pk_dil.as_bytes()         // 1952 bytes (Dilithium-3 pubkey)
    || pk_x25.as_bytes()         // 32 bytes
    || pk_kyber.as_bytes()       // 1568 bytes (Kyber-1024 pubkey)
);
// node_id: 32 bytes = 256 bits = position in Kademlia keyspace
```

Listing 1 — Node identity generation

3.2 Self-Certification Verification

Anyone who receives a NodeDescriptor can verify the node's identity in $O(1)$ time, without any network request:

```
VERIFY(descriptor):
1. Recompute: claimed_id = BLAKE3('phantom-node-v1' || pk_ed || pk_dil ||
pk_x25 || pk_kyber)
2. Assert:    claimed_id == descriptor.node_id
3. Verify:    Ed25519.Verify(pk_ed, descriptor.canonical_bytes(),
descriptor.sig_ed)
4. Verify:    Dilithium3.Verify(pk_dil, descriptor.canonical_bytes(),
descriptor.sig_dil)
5. Assert:    descriptor.epoch == current_epoch() OR current_epoch() - 1

If all 5 checks pass: descriptor is authentic. No authority consulted.
```

3.3 Key Rotation

Long-term signing keys (Ed25519, Dilithium-3) are permanent — they define the node's identity. KEM keys (X25519, Kyber-1024) rotate every epoch. New KEM public keys are published in each epoch's NodeDescriptor, signed by the permanent signing keys. This ensures forward secrecy: past session keys cannot be derived even if current KEM keys are compromised.

4. Sybil Resistance — Dual Crypto-Puzzle Admission

Phantom Protocol inherits and extends the S/Kademlia dual-puzzle scheme. The fundamental insight is that if generating a valid node identity costs meaningful computational work, then creating thousands of Sybil identities becomes proportionally expensive.

4.1 Static Puzzle — Identity Generation Cost

The static puzzle is solved once per node identity. It is embedded in the identity generation process: the node must find an Ed25519 key pair whose public key hashes (via BLAKE3) to a value with at least C1 leading zero bits.

```
Static Puzzle:
Find (sk_ed, pk_ed) such that: BLAKE3(pk_ed)[0..C1] == 0...0
Expected attempts: 2^C1
C1 = 20 bits → ~1,048,576 hash evaluations per identity
On modern hardware: ~50ms per identity
Creating 1,000 identities: ~50 seconds, ~50 CPU-minutes
Creating 1,000,000 identities: ~14 CPU-hours

Adversary with 1,000 cores creating Sybils: 14 CPU-hours / 1,000 = ~50 seconds
→ C1 alone is insufficient. Dynamic puzzle provides the epoch-binding
constraint.
```

4.2 Dynamic Puzzle — Epoch-Bound Admission

The dynamic puzzle binds a node to a specific epoch. It must be solved freshly every epoch (every hour). Even an adversary who pre-computed millions of static-puzzle solutions cannot activate them all simultaneously — each requires a fresh dynamic-puzzle solution tied to the current epoch.

```
Dynamic Puzzle:
Find nonce such that: BLAKE3(node_id XOR nonce || epoch)[0..C2] == 0...0
Expected attempts: 2^C2
C2 = 16 bits → ~65,536 evaluations per epoch per identity
On modern hardware: ~3ms per identity per epoch

Combined constraint:
To activate N identities per epoch, adversary must:
- Solve N static puzzles (one-time): N * 2^C1 hashes
- Solve N dynamic puzzles per hour: N * 2^C2 hashes/epoch

To flood 10,000 Sybils per hour on a 100,000-node network (10%):
Static: 10,000 * 2^20 = ~10 billion hashes (one-time, ~3 days on 1 core)
Dynamic: 10,000 * 2^16 = ~655M hashes/hour (ongoing, ~11 CPU-minutes/hour)
→ Meaningful deterrent without requiring trusted issuance.
```

4.3 Admission Token

The combination of static and dynamic puzzle solutions forms an admission token, included in every NodeDescriptor:

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct AdmissionToken {
    pub static_nonce: [u8; 8],    // nonce that solves static puzzle
    pub dynamic_nonce: [u8; 8],   // nonce that solves dynamic puzzle
    pub epoch:        u32,        // epoch this token is valid for
}

impl AdmissionToken {
    pub fn verify(&self, node_id: &[u8; 32], pk_ed: &[u8; 32]) -> bool {
        // Verify static puzzle
        let static_hash = blake3::hash(pk_ed);
        let static_ok = static_hash.leading_zeros() >= C1_BITS;

        // Verify dynamic puzzle
        let mut dyn_input = [0u8; 48];
        for i in 0..32 { dyn_input[i] = node_id[i] ^ self.dynamic_nonce[i % 8]; }
        dyn_input[32..36].copy_from_slice(&self.epoch.to_be_bytes());
        let dyn_hash = blake3::hash(&dyn_input);
        let dynamic_ok = dyn_hash.leading_zeros() >= C2_BITS;

        // Verify epoch is current
        let epoch_ok = self.epoch == current_epoch();

        static_ok && dynamic_ok && epoch_ok
    }
}
```

Listing 2 — Admission token verification

4.4 Adaptive Difficulty

Network difficulty (C1 and C2) is adjusted each epoch based on observed admission rate. If more than MAX_ADMISSIONS new nodes are admitted in an epoch, C2 increments by 1. If fewer than MIN_ADMISSIONS, C2 decrements by 1. C1 is fixed at genesis and changes only via protocol upgrade. This mirrors Bitcoin's difficulty adjustment but operates per-epoch rather than per-block.

```
const C1_BITS: u32 = 20;           // static - fixed at genesis
const C2_BITS_DEFAULT: u32 = 16;  // dynamic - adjusts each epoch
const MAX_ADMISSIONS_PER_EPOCH: u32 = 10_000;
const MIN_ADMISSIONS_PER_EPOCH: u32 = 100;

fn adjust_difficulty(observed_admissions: u32, current_c2: u32) -> u32 {
    if observed_admissions > MAX_ADMISSIONS_PER_EPOCH {
        (current_c2 + 1).min(28) // cap at 28 bits
    } else if observed_admissions < MIN_ADMISSIONS_PER_EPOCH {
        current_c2.saturating_sub(1).max(10) // floor at 10 bits
    } else {
        current_c2
    }
}
```

5. NodeDescriptor Record Format

The NodeDescriptor is the fundamental unit of the Phantom-DPKI. It is the record that every node publishes to the DHT, that every other node retrieves to find relays, and that is verified before any packet is routed through a node.

5.1 Full Structure

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct NodeDescriptor {
    // — Identity —
    pub node_id: [u8; 32], // BLAKE3(pubkeys) — DHT key
    pub pk_ed25519: [u8; 32], // Ed25519 signing pubkey (permanent)
    pub pk_dilithium: [u8; 1952], // Dilithium-3 signing pubkey (permanent)

    // — Session keys (rotate every epoch) —
    pub pk_x25519: [u8; 32], // X25519 KEM pubkey
    pub pk_kyber: [u8; 1568], // Kyber-1024 KEM pubkey

    // — Admission —
    pub admission: AdmissionToken, // dual PoW proof

    // — Network —
    pub onion_address: [u8; 32], // BLAKE3(pk_ed || pk_kyber)[:32] for hidden
    svc
    pub quic_port: u16, // public QUIC port (usually 443)
    pub capacity_tier: CapacityTier, // Full | Light | Mobile
    pub mix_layer: u8, // 0=guard, 1=middle, 2=exit (rotates per
    epoch)

    // — Epoch —
    pub epoch: u32, // epoch this descriptor is valid for
    pub published_at: u64, // Unix timestamp of DHT publication
    pub expires_at: u64, // published_at + 3600 seconds

    // — Proof of shuffle capability —
    pub zk_capability: ZkCapability, // BLS12-381 verification key ref

    // — Reputation (gossip-accumulated) —
    pub valid_proof_count: u32, // ZK proofs verified by peers this epoch
    pub uptime_score: u8, // 0-255, smoothed availability

    // — Signatures (both required) —
    pub sig_ed25519: [u8; 64], // Ed25519 signature over canonical bytes
    pub sig_dilithium: [u8; 3293], // Dilithium-3 signature over canonical bytes
}

// Total descriptor size: ~7.5 KB (dominated by Dilithium-3 signature: 3293 bytes)
// Compressed (zstd): ~4.5 KB average
```

Listing 3 — NodeDescriptor full structure

5.2 Canonical Bytes — What is Signed

The signed payload covers every field except the two signature fields themselves, in the following fixed-length serialisation order. The order is deterministic and version-locked — any change requires a protocol version increment:

```
canonical = b'phantom-descriptor-v1'
    || node_id           (32 bytes)
    || pk_ed25519        (32 bytes)
    || pk_dilithium       (1952 bytes)
    || pk_x25519         (32 bytes)
    || pk_kyber           (1568 bytes)
    || admission.to_bytes() (20 bytes)
    || onion_address      (32 bytes)
    || quic_port.to_be()  (2 bytes)
    || capacity_tier as u8 (1 byte)
    || mix_layer          (1 byte)
    || epoch.to_be()      (4 bytes)
    || published_at.to_be() (8 bytes)
    || expires_at.to_be() (8 bytes)

sig_ed25519 = Ed25519.Sign(sk_ed, BLAKE3(canonical))
sig_dilithium = Dilithium3.Sign(sk_dil, BLAKE3(canonical))
```

5.3 Capacity Tiers

Tier	Min Bandwidth	Min CPU	Proof Generation	Max Batch Size	Roles
Full	100 Mbps	8 cores	Yes	512	Guard, Middle, Exit, Hidden Service
Light	10 Mbps	4 cores	Yes	64	Middle, Guard
Mobile	1 Mbps	Any	No (verify only)	N/A	Client only, contributes cover traffic

Table 2 — Node capacity tiers and roles

6. DHT Architecture — Phantom-Kademlia

Phantom-DPKI is built on a hardened variant of Kademlia, incorporating the security improvements of S/Kademlia and several Phantom-specific extensions. It is implemented using libp2p's Kademlia module with custom validation logic and the Phantom-specific record types.

6.1 Keyspace and Distance Metric

Phantom-Kademlia uses a 256-bit keyspace — the output space of BLAKE3. The XOR distance metric is inherited unchanged from standard Kademlia. Node IDs are 32-byte BLAKE3 digests, placing them uniformly in the keyspace by the hash function's pseudorandomness.

```
distance(a, b) = a XOR b    (bitwise, interpreted as unsigned 256-bit integer)
keyspace = {0, 1}^256
replication factor k = 20   (Kademlia standard)
lookup concurrency  $\alpha$  = 3 (parallel lookup paths)
disjoint paths d = 5        (eclipse resistance, see §8)
```

6.2 Routing Table — k-Buckets

Each node maintains 256 k-buckets, one per bit of the keyspace. Each k-bucket holds up to $k=20$ NodeDescriptor references for nodes at that XOR distance range. Kademlia's standard preference for long-lived nodes is preserved — this naturally favours stable honest nodes over newly-created Sybils, which by definition haven't been online long.

```
pub struct KBucket {
    entries:    Vec<NodeRef>,    // up to k=20 entries, LRU order
    replacement: Vec<NodeRef>,    // replacement cache
}

pub struct NodeRef {
    pub node_id:    [u8; 32],
    pub quic_addr:  SocketAddr, // IP:port for QUIC connection
    pub last_seen:  Instant,     // for LRU eviction
    pub valid_epoch: u32,        // epoch of last valid descriptor seen
    pub trust_score: u8,         // local reputation (0=unknown, 255=verified)
}

// ADMISSION RULE: A node is only added to a k-bucket if:
// 1. Its NodeDescriptor passes full verification (§3.2)
// 2. Its AdmissionToken is valid for the current epoch (§4.3)
// 3. We have successfully completed a QUIC handshake with it
// 4. It has responded to at least one FIND_NODE RPC
```

6.3 Phantom-Kademlia RPCs

Phantom-Kademlia defines six RPCs, transmitted as encrypted QUIC streams with Sphinx⁺-like header structure:

RPC	Direction	Purpose	Auth Required
PING	→ node	Liveness check + epoch verification	Valid descriptor
FIND_NODE	→ node	Return k closest known nodes to target_id	Valid descriptor
FIND_DESC	→ node	Return NodeDescriptor for a specific node_id	Valid descriptor
STORE_DESC	→ k closest nodes	Publish own NodeDescriptor	Own descriptor + valid admission
STORE_PROOF	→ k closest nodes	Publish ZK shuffle proof	Own descriptor
STORE_COMMIT	→ k closest nodes	Publish batch commitment (C_in or C_out)	Own descriptor

Table 3 — Phantom-Kademlia RPC set

All RPCs are transmitted over QUIC streams, authenticated by the sender's Ed25519 signature over the RPC payload. Nodes that send unauthenticated or invalidly-signed RPCs are immediately added to a temporary block list and removed from k-buckets if present.

6.4 Storage Namespaces

The DHT keyspace is partitioned into namespaces via key prefixing. Records in different namespaces can coexist at the same XOR distance without collision:

Namespace	Key Construction	Value Type	TTL
Descriptor	BLAKE3('desc' node_id epoch)	NodeDescriptor	1 epoch
Batch Commit	BLAKE3('commit' batch_id epoch)	BatchCommitment	1 epoch
ZK Proof	BLAKE3('proof' batch_id epoch)	ShuffleProof	1 epoch
Ejection	BLAKE3('eject' node_id epoch)	EjectionNotice	2 epochs
Hidden Svc	BLAKE3('hs' onion_addr epoch)	IntroductionPoints	1 epoch
Epoch Info	BLAKE3('epoch' epoch_number)	EpochParameters	1 epoch

Table 4 — DHT storage namespaces

7. Record Lifecycle — Publish, Replicate, Expire

7.1 Initial Publication

```

async fn publish_descriptor(&self) {
    // 1. Build descriptor for current epoch
    let desc = self.build_descriptor(current_epoch());

    // 2. Find k closest nodes to our node_id
    let closest = self.dht.find_node(self.node_id).await;

    // 3. Send STORE_DESC to all k closest nodes (parallel)
    let futs = closest.iter().map(|peer| self.rpc_store_desc(peer, &desc));
    futures::future::join_all(futs).await;

    // 4. Store locally for serving FIND_DESC requests
    self.local_store.insert(descriptor_key(self.node_id), desc);

    // 5. Schedule re-publication at 75% of epoch duration
    let republish_at = epoch_end() - Duration::from_secs(900);
    self.scheduler.schedule(republish_at, Task::RepublishDescriptor);
}

```

7.2 Replication

Each node stores descriptors for the k nodes closest to its own ID. When a new node joins and becomes a close neighbour, existing nodes proactively send it descriptors they hold. This ensures k -fold replication of every descriptor across the k nearest nodes, providing fault tolerance against node churn.

The replication factor $k=20$ means a descriptor survives the simultaneous departure of up to 19 of its 20 replica holders — an extremely conservative guarantee suited to high-churn anonymous networks.

7.3 Expiration

All records carry an `expires_at` Unix timestamp. Nodes purge expired records from local storage without any explicit delete message. This means:

- ▶ A node that is taken offline simply disappears from the DHT within one epoch. No explicit deregistration or announcement is needed or possible.
- ▶ A compromised node descriptor becomes useless after its epoch ends. An adversary who obtains a descriptor cannot use it in future epochs.
- ▶ There is no global record deletion operation — no record can be suppressed or censored by a third party.

7.4 Republication

Nodes must republish their descriptor before epoch expiry. Republication triggers a fresh dynamic puzzle solve and re-signs the descriptor with updated KEM public keys. The DHT key for the new epoch is different from the previous epoch's key (due to the epoch number in the key derivation), so old and new descriptors coexist briefly during epoch transitions without conflict.

8. Multi-Path Eclipse-Resistant Lookup

Standard Kademlia lookup is vulnerable to eclipse attacks: an adversary who controls nodes in the routing path can return incorrect results, diverting the lookup to adversarial nodes. S/Kademlia's core contribution is multi-path disjoint lookup: running d independent lookup paths in parallel and accepting a result only if a quorum of paths agree.

8.1 Standard Kademlia Lookup (Vulnerable)

```
// VULNERABLE: single path, single point of manipulation
fn lookup_standard(target: NodeId) -> Option<NodeDescriptor> {
    let mut candidates = self.k_closest(target); // from local routing table
    loop {
        let next = candidates.closest_unqueried();
        let response = self.rpc_find_node(next, target);
        candidates.add(response.nodes);
        if no_improvement { break; }
```

```

    }
    candidates.first() // adversary can control this result
}

```

8.2 Phantom Multi-Path Lookup

```

const DISJOINT_PATHS: usize = 5; // d = 5 independent paths
const QUORUM: usize = 3; // need 3/5 paths to agree

async fn lookup_phantom(target: NodeId) -> Option<NodeDescriptor> {
    // 1. Choose d disjoint starting points from local routing table
    // 'Disjoint' means: no two paths share an intermediate node
    let seeds: Vec<Vec<NodeRef>> = self.d_disjoint_seeds(target, DISJOINT_PATHS);

    // 2. Run d lookup paths in parallel
    let path_results = futures::future::join_all(
        seeds.iter().map(|seed| self.single_path_lookup(target, seed))
    ).await;

    // 3. Collect results and verify each
    let verified: Vec<NodeDescriptor> = path_results
        .into_iter()
        .flatten()
        .filter(|desc| desc.verify_full()) // full cryptographic verification
        .collect();

    // 4. Accept if QUORUM paths return the same result (by node_id)
    let counts = count_by(verified, |d| d.node_id);
    counts.into_iter()
        .filter(|(_, count)| *count >= QUORUM)
        .map(|(desc, _)| desc)
        .next()
}

```

Listing 4 — Multi-path eclipse-resistant lookup

Eclipse Resistance Analysis

With $d=5$ disjoint paths and $quorum=3$, an adversary must control at least one node on 3 out of 5 independent paths to manipulate a lookup result. S/Kademlia's analysis shows that with 20% adversarial nodes and $d=5$, the probability of a successful eclipse attack is below 0.0001%. This is compatible with the Phantom threat model at the DHT layer. The mix network's own ZK proof system provides a second independent layer of protection against compromised relay nodes.

9. Gossip Ejection Protocol

The gossip ejection protocol enables the network to propagate misbehaviour proofs in seconds, immediately reducing the impact of a node that produces an invalid shuffle proof, equivocates, or exhibits other detectable malicious behaviour.

9.1 Ejectable Offences

Offence	Detection Method	Evidence
Invalid shuffle proof	ZK proof verification fails	(C_{in}, C_{out}, π) — verifiable by anyone
Proof not published	Batch window expired, no proof in DHT	$(batch_id, epoch, node_id) + timestamp$
Equivocation	Two different C_{in} for same $batch_id$	Both C_{in} records + timestamps
Descriptor forgery	Signature verification fails	Forged descriptor bytes
Replay attack	Packet seen in replay cache	$(packet_alpha, epoch)$
Epoch violation	Descriptor for wrong epoch	Descriptor bytes

Table 5 — Ejectable offences and their evidence

9.2 Ejection Notice Format

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct EjectionNotice {
    pub offender_id: [u8; 32], // node_id of misbehaving node
    pub epoch: u32,
    pub offence: OffenceType,
    pub evidence: Vec<u8>, // serialised evidence (offence-specific)
    pub reporter_id: [u8; 32], // node_id of detecting node
    pub timestamp: u64,
    pub sig_ed: [u8; 64], // reporter's Ed25519 signature
    pub sig_dil: [u8; 3293], // reporter's Dilithium-3 signature
}

// Evidence for invalid shuffle proof:
pub struct InvalidProofEvidence {
    pub c_in_hash: [u8; 32], // BLAKE3(C_in) — fetch full C_in from DHT
    pub c_out_hash: [u8; 32], // BLAKE3(C_out)
    pub proof_hash: [u8; 32], // BLAKE3( $\pi$ )
    // Any verifier can fetch these from DHT and re-run verification
}
```

9.3 Gossip Propagation

```
async fn handle_ejection_notice(&self, notice: EjectionNotice) {
    // 1. Verify reporter's signatures
    if !notice.verify_signatures() { return; } // silently drop

    // 2. Verify evidence (re-run the verification that failed)
    if !self.verify_ejection_evidence(&notice).await { return; }

    // 3. Add offender to local ban list
    self.ban_list.insert(notice.offender_id, BanEntry {
        epoch: notice.epoch,
        reason: notice.offence.clone(),
        expires: epoch_end() + EPOCH_DURATION, // 2-epoch ban
    });

    // 4. Remove from all k-buckets immediately
```

```

self.routing_table.remove(notice.offender_id);

// 5. Store ejection notice in DHT (persists for 2 epochs)
let key = ejection_key(notice.offender_id, notice.epoch);
self.dht.store(key, notice.clone()).await;

// 6. Forward to all peers (gossip fan-out: 3)
//    libp2p GossipSub handles deduplication and propagation
self.gossip.publish("phantom/ejection/v1", notice.encode());
}

// Target propagation time: < 5 seconds to reach 95% of network
// libp2p GossipSub provides logarithmic propagation complexity

```

Listing 5 — Ejection notice handling and gossip propagation

9.4 False Ejection Resistance

An adversary could attempt to falsely eject an honest node by publishing a fabricated ejection notice. This is prevented by the evidence requirement: every ejection notice must include evidence that any verifier can independently check. A notice claiming invalid shuffle proof must reference a (C_{in}, C_{out}, π) triple that actually fails verification — which requires the adversary to produce a genuine failure, i.e., to control the node being ejected.

False ejection notices with unfalsifiable claims (e.g., 'proof not published') are mitigated by the timestamp and DHT-availability check: verifiers check whether the proof was present in the DHT within the batch window before accepting a 'not published' ejection.

10. Epoch Rotation and Forward Secrecy

10.1 Epoch Parameters

Parameter	Value	Rationale
Epoch duration	3600 seconds (1 hour)	Balance: republication cost vs. forward secrecy window
Epoch start	Unix time rounded to nearest hour	Deterministic, no coordination needed
Overlap period	120 seconds	Old and new descriptors coexist during transition
Key rotation	KEM keys only (Ed/Dil permanent)	Identity stable; session keys forward-secret
PoW re-solve	Dynamic puzzle per epoch	Prevents pre-computed Sybil activation
DHT key change	epoch number in key derivation	Old/new records don't collide
Reputation reset	Partial (50% weight carry-forward)	Prevents reputation poisoning

10.2 Epoch Transition Timeline

```

T - 900s:  Begin solving dynamic puzzle for next epoch
T - 300s:  Generate new KEM keypair (X25519 + Kyber-1024)
T - 120s:  Publish next-epoch NodeDescriptor to DHT (with next epoch number)
           Both current and next descriptors valid during overlap
T + 0s:    New epoch begins
           Current-epoch descriptors still served for 120s (overlap)
T + 120s:  Purge previous-epoch descriptors from local store
           Any node not yet republished is unreachable (treated as offline)
T +3600s:  Next epoch begins — cycle repeats

```

10.3 Forward Secrecy Guarantee

Because KEM keys rotate every epoch, an adversary who compromises a node's current KEM private keys cannot decrypt traffic from previous epochs. Each epoch's session keys are derived from ephemeral key exchanges using that epoch's KEM keys — once the epoch ends and the KEM keys are rotated, past sessions cannot be decrypted even with full node compromise.

Long-term signing keys (Ed25519, Dilithium-3) do not rotate — they define node identity. Compromise of signing keys allows impersonation of the node's identity but does not compromise past or future session traffic (since session keys are KEM-derived, not signing-key-derived).

11. Bootstrap Without Seed Nodes

Traditional P2P networks require at least one hardcoded seed node or trusted bootstrap server. This creates a legal attack surface: compel the seed node operator and the network cannot grow. Phantom Protocol eliminates this entirely through a three-layer bootstrap mechanism.

11.1 Layer 1: Local Network (mDNS)

On first startup, the node broadcasts a mDNS announcement on the local network. If any other Phantom node is present on the same LAN (home network, university, office), they discover each other immediately without any internet contact. This is the zero-external-dependency bootstrap path — functional even if the entire internet is blocked.

```

// mDNS service name: _phantom._quic.local
// Announcement payload: node_id (hex) + QUIC port
// Verified: recipient performs full NodeDescriptor verification before adding to
// routing table
let mdns = libp2p::mdns::tokio::Behaviour::new(
    libp2p::mdns::Config::default(),
    local_peer_id,
)??;

```

11.2 Layer 2: DNS Bootstrap (No Trusted Server)

Phantom Protocol publishes `node_id` values encoded as DNS TXT records under a distributed set of domains. Any Phantom contributor can operate a bootstrap DNS record. The critical property: DNS records contain only `node_ids` — no IP addresses. The IP is discovered via the DHT lookup using the `node_id`.

This means: a DNS server can be compelled to return no records, or wrong records — but it cannot return a valid fake `node_id`, because `node_ids` are self-certifying (§3.1). Fake `node_ids` fail the admission token check and are immediately discarded.

```
// DNS bootstrap format:
// TXT_phantom-bootstrap.example.com:
//   v=phantom1 nid=<hex node_id> epoch=<epoch>

// Client verifies: H(pk_ed || pk_dil || pk_x25 || pk_kyber) == node_id
// A DNS operator cannot forge a valid node_id without solving the static puzzle

async fn dns_bootstrap(resolver: &Resolver) -> Vec<NodeId> {
    let domains = PHANTOM_BOOTSTRAP_DOMAINS; // 20+ domains by different operators
    let futs = domains.iter().map(|d| resolver.txt_lookup(d));
    let records = futures::future::join_all(futs).await;
    records.into_iter().flatten()
        .filter_map(|txt| parse_bootstrap_record(txt))
        .collect()
}
```

11.3 Layer 3: Genesis CID (IPFS-Pinned)

The genesis NodeDescriptor set is content-addressed and pinned on IPFS. Any node can fetch it via any IPFS gateway or the IPFS network itself. The content address (CID) is hardcoded in the Phantom Protocol binary — but unlike a hardcoded IP address, a CID is a hash: it is immutable, verifiable, and accessible from any IPFS node worldwide.

```
// Hardcoded in binary (NOT a trusted server — it's a content hash):
const GENESIS_CID: &str = "bafybeig..."; // IPFS CIDv1, BLAKE3-based

// Genesis file contains: Vec<NodeDescriptor> for epoch 1
// Anyone can verify: BLAKE3(genesis_bytes) == CID_content_hash
// Anyone can serve: any IPFS node can pin and serve this file
// Adversary cannot forge: the content hash is in the binary

async fn genesis_bootstrap() -> Vec<NodeDescriptor> {
    // Try IPFS gateways (any of them, order randomised)
    for gateway in IPFS_GATEWAYS.choose_multiple(&mut rng, 3) {
        if let Ok(data) = fetch_ipfs(gateway, GENESIS_CID).await {
            if blake3(&data) == genesis_hash() { // verify content
                return serde_json::from_slice(&data).unwrap_or_default();
            }
        }
    }
    vec![] // genesis bootstrap failed — try other layers
}
```

11.4 Bootstrap Fallback Sequence

```

async fn bootstrap(&self) {
    // Try all three layers simultaneously
    let (mdns, dns, genesis) = tokio::join!(
        self.mdns_discover(),
        self.dns_bootstrap(),
        genesis_bootstrap(),
    );

    let mut seeds: Vec<NodeDescriptor> = [mdns, dns, genesis]
        .into_iter().flatten()
        .filter(|d| d.verify_full()) // reject any invalid descriptors
        .collect();

    if seeds.is_empty() {
        // All three layers failed. Network may be censored.
        // User must provide a peer address manually (out-of-band).
        log::warn!("Bootstrap failed. Provide a peer address via --peer flag.");
        return;
    }

    // Perform self-lookup to populate routing table
    for seed in seeds.iter().take(3) {
        self.routing_table.add(seed.clone());
    }
    self.dht.find_node(self.node_id).await; // self-lookup
    self.publish_descriptor().await;
}

```

12. DHT as Multi-Purpose Registry

The Phantom-DPKI DHT serves as the universal registry for all Phantom Protocol state. Beyond node discovery, it stores the cryptographic artifacts defined in Technical Specification № 2, as well as the hidden service introduction points defined in Technical Specification № 1.

12.1 Complete Record Type Registry

Record Type	Stored By	Retrieved By	Used For
NodeDescriptor	Node (self)	Any node	Routing, key material, mix layer assignment
BatchCommitment C _{in}	Mix node	Any verifier	ZK proof input
BatchCommitment C _{out}	Mix node	Any verifier	ZK proof output
ShuffleProof π	Mix node	Any verifier	Verify correct mixing
EjectionNotice	Any detector	Any node	Routing table update, ban
IntroductionPoints	Hidden service	Any client	Hidden service connection
EpochParameters	Network genesis	Any node	Difficulty parameters, epoch timing

RepairAdvertisement	Full nodes	Light/mobile nodes	Routing table repair after partition
---------------------	------------	--------------------	--------------------------------------

Table 6 — Complete DHT record type registry

12.2 Privacy Preservation in DHT Lookups

A node looking up another node's descriptor reveals its interest to the k closest nodes it queries. In a standard Kademlia network this leaks information: 'node A is interested in connecting to node B.' Phantom-DPKI mitigates this through two mechanisms:

- ▶ Private lookups: the lookup key is hashed with a per-session random salt before being submitted to the DHT. Only the final k nodes (who return the actual record) see the unhashed lookup key.
- ▶ Circuit pre-building: nodes pre-fetch descriptors for a random set of nodes each epoch, making their actual circuit-building lookups indistinguishable from the background noise of random descriptor fetches.

13. Rust Implementation Guide

13.1 Crate Dependencies

Component	Crate	Notes
Kademlia DHT	libp2p (kad)	Use custom record validator for NodeDescriptor
Gossip propagation	libp2p (gossipsub)	Topic: phantom/ejection/v1
mDNS discovery	libp2p (mdns)	Service: _phantom._quic.local
QUIC transport	quinn	All RPC communication
Ed25519	ed25519-dalek	Signing and verification
Dilithium-3	pqcrypto-dilithium	PQ signing
Kyber-1024	pqcrypto-kyber	PQ KEM
X25519	x25519-dalek	Classical KEM
BLAKE3	blake3	All hashing
DNS resolution	hickory-resolver	Bootstrap DNS lookup
IPFS client	iroh	Genesis CID fetch
Serialisation	serde + postcard	No-std compatible binary format
Zeroisation	zeroize	Secret key cleanup on drop

13.2 Module Structure

```
phantom-dpki/
├── src/
│   ├── identity.rs           // Key generation, node_id derivation, self-cert verify
│   ├── puzzle.rs            // Static + dynamic PoW puzzle, adaptive difficulty
│   ├── descriptor.rs        // NodeDescriptor struct, signing, canonical bytes
│   └── dht/
│       ├── mod.rs           // Phantom-Kademlia wrapper around libp2p kad
│       ├── buckets.rs       // k-bucket management, admission rules
│       ├── lookup.rs        // Multi-path disjoint lookup (d=5, quorum=3)
│       ├── store.rs         // Namespace-aware record storage and TTL
│       ├── rpc.rs           // PING, FIND_NODE, FIND_DESC, STORE_* RPCs
│       ├── gossip.rs        // EjectionNotice format, propagation, verification
│       ├── bootstrap.rs     // mDNS + DNS + genesis CID bootstrap
│       ├── epoch.rs         // Epoch timing, rotation, key refresh
│       └── lib.rs           // Public API: PhantomDPKI struct
├── tests/
│   ├── integration/        // Multi-node tests (mini-network in tokio)
│   └── unit/               // Per-module unit tests
└── benches/               // criterion benchmarks for puzzle, lookup, verify
```

13.3 Core API

```
pub struct PhantomDPKI {
    node: PhantomNode,
    dht: PhantomKademlia,
    gossip: GossipLayer,
    epoch: EpochManager,
}

impl PhantomDPKI {
    /// Start the DPKI layer. Generates identity, solves puzzles, bootstraps,
    publishes.
    pub async fn start(config: DPKIConfig) -> Result<Self, DPKIError>;

    /// Resolve a node_id to its current NodeDescriptor.
    pub async fn resolve(&self, node_id: NodeId) -> Option<NodeDescriptor>;

    /// Report a misbehaving node with evidence. Triggers gossip ejection.
    pub async fn report_misbehaviour(&self, offender: NodeId,
                                     offence: OffenceType,
                                     evidence: Vec<u8>) -> Result<(), DPKIError>;

    /// Publish a record to the DHT (batch commitment, ZK proof, etc.)
    pub async fn store(&self, namespace: Namespace,
                      key: [u8; 32],
                      value: Vec<u8>) -> Result<(), DPKIError>;

    /// Retrieve a record from the DHT.
    pub async fn fetch(&self, namespace: Namespace,
                      key: [u8; 32]) -> Option<Vec<u8>>;

    /// Return k random nodes at a given mix layer for circuit building.
    pub async fn sample_mix_nodes(&self, layer: u8, k: usize) -> Vec<NodeDescriptor>;

    /// Check if a node is currently banned (ejected).
    pub fn is_banned(&self, node_id: NodeId) -> bool;}
```

Listing 6 — PhantomDPKI public API

14. Attack Analysis

Attack	Mechanism	Phantom-DPKI Resistance	Residual Risk
Sybil (mass identity)	Create many fake nodes to flood network	Dual PoW: each identity costs ~50ms + 3ms/epoch. 10,000 Sybils = 3+ CPU-days (one-time)	Well-resourced adversary can still field thousands of Sybils; mix network ZK proofs provide the second line
Eclipse (routing manipulation)	Control nodes in lookup path to divert queries	d=5 disjoint paths, quorum=3: adversary needs 3 of 5 independent paths controlled	~0.0001% success rate at 20% adversary nodes (S/Kademlia analysis)
Directory compulsion	Compel authority to inject/remove nodes	No directory authorities exist	None at PKI layer
Poisoning (fake descriptors)	Publish descriptors claiming to be other nodes	Self-certification: node_id = H(pubkeys). Cannot forge without solving PoW	None — mathematically prevented
Churn flooding	Repeatedly join/leave to destabilize routing	k-bucket prefers long-lived nodes; new nodes only replace offline ones	Sustained churn attack is expensive due to PoW per-epoch
Replay (old descriptors)	Re-publish old valid descriptors to divert traffic	Epoch number in DHT key: old descriptors stored at different keys	None — epoch-keyed
False ejection	Fabricate ejection notices for honest nodes	Evidence requirement: verifiable by anyone; fake evidence fails independently	Requires real misbehaviour to produce valid evidence
Bootstrap poisoning	Serve false data at DNS/IPFS bootstrap	DNS: node_ids self-certifying, fake ones fail PoW check. IPFS: CID is content hash	None — both layers are self-verifying

Table 7 — Attack analysis for Phantom-DPKI

15. Open Problems

Phantom-DPKI is honest about the problems it has not yet solved:

- Sybil resistance at large scale: the dual-puzzle scheme deters casual Sybil attacks but does not prevent a nation-state adversary with significant compute resources from fielding millions of Sybils over time. Research into STARK-based proof-of-unique-identity (without revealing identity) is ongoing.
- DHT lookup privacy: the private lookup mechanism (random-salt hashing + background noise) reduces but does not eliminate the information leaked by DHT queries. A fully

private DHT lookup requires oblivious RAM constructions, which are too expensive for real-time operation at present.

- ▶ Adversarial epoch timing: an adversary who can observe epoch transitions may be able to correlate node activity across epochs even with key rotation, if the node's IP address is consistent. The pluggable transport layer (Sphinx⁺ over QUIC, mimicking HTTPS) partially mitigates this.
- ▶ Incentive for honest DHT storage: nodes store others' records as a side effect of being in the DHT. There is no explicit reward for storing records reliably. In a network with many Mobile-tier nodes (who contribute minimally), record availability may degrade. A lightweight reputation mechanism for storage reliability is under consideration.
- ▶ Genesis CID update: the hardcoded genesis CID creates a one-time dependency on a specific IPFS content address. Protocol upgrades require a new genesis CID distributed via software update — a centralisation pressure. A bootstrapless upgrade mechanism (propagated via the live network) is a future design goal.

16. References

1. [MM02] Maymounkov, P., Mazières, D. — Kademlia: A Peer-to-Peer Information System Based on the XOR Metric. IPTPS 2002.
2. [BM07] Baumgart, I., Mies, S. — S/Kademlia: A Practicable Approach Towards Secure Key-Based Routing. IEEE ICPADS 2007.
3. [DG09] Danezis, G., Goldberg, I. — Sphinx: A Compact and Provably Secure Mix Format. IEEE S&P 2009.
4. [NIST203] NIST — CRYSTALS-Kyber (ML-KEM), FIPS 203. 2024.
5. [NIST204] NIST — CRYSTALS-Dilithium (ML-DSA), FIPS 204. 2024.
6. [IPFS-KAD] Protocol Labs — IPFS Kademlia DHT Specification. specs.ipfs.tech/routing/kad-dht/.
7. [LP2P] libp2p Contributors — libp2p Specifications. github.com/libp2p/specs.
8. [UDHTS] Urdaneta, G., Pierre, G., Steen, M.V. — A Survey of DHT Security Techniques. ACM Computing Surveys 2011.
9. [SE17] Sohliloquies — Resisting Sybil Attacks in Distributed Hash Tables. 2017.
10. [ZBV24] Zhang, Y., Venkatakrishnan, S.B. — Kadabra: Adapting Kademlia for the Decentralized Web. Financial Cryptography 2024.

— Phantom Protocol Technical Specification № 3 — End of Document —